

# University of Central Punjab



## Project Proposal

**A Project Proposal For [Big Data Analysis / Bachelor's of Artificial Intelligence (AI)]**



Intelligent Symptom  
Analysis & Healthcare  
Advisory Platform

### **Submitted To:**

Mr. Zishan Hussain Chuhan

Dept Of Applied Computing and Technology

### **Submitted By:**

Ahmed Ali (L1F23BSAI0092)

M.Qaim Hussnain (L1F23BSAI0077)

Noman Tariq Butt (L1F23BSAI0095)

# Table of Contents

MediSpark — Intelligent Symptom Analysis & Healthcare Advisory Platform

|                                                                   |    |
|-------------------------------------------------------------------|----|
| <b>1. Executive Summary</b>                                       | 1  |
| 1.1 Key Capabilities                                              | 1  |
| <b>2. System Architecture</b>                                     | 2  |
| <b>3. Technology Stack</b>                                        | 3  |
| <b>4. Project Structure</b>                                       | 4  |
| <b>5. Data Flow</b>                                               | 5  |
| <b>6. Smart Features Implementation</b>                           | 6  |
| 6.1 Severity Score (0–100)                                        | 6  |
| 6.2 Risk Assessment Logic                                         | 7  |
| 6.3 Kafka Topics Architecture                                     | 7  |
| 6.4 Roman Urdu / English NLP                                      | 8  |
| <b>7. 4-Week Development Plan</b>                                 | 9  |
| Week 1 — Foundation & Setup (Days 1–7)                            | 9  |
| Week 2 — Core AI Engine (Days 8–14)                               | 10 |
| Week 3 — Smart Features + PySpark,Tools&Tech (Days 15–21)         | 11 |
| Week 4 — Polish, Integration & Deployment,Challanges (Days 22–28) | 12 |
| <b>8. Datasets</b>                                                | 13 |
| <b>9. Key Dependencies (requirements.txt)</b>                     | 14 |
| <b>10. Project Summary</b>                                        | 15 |

# 1. Executive Summary

---

MediSpark is a full-stack AI-powered healthcare web application built with Flask that accepts patient symptoms, predicts diseases using Machine Learning and Retrieval-Augmented Generation (RAG), suggests medicines, assesses risk levels, and recommends diagnostic tests for high-risk cases. The platform supports a bilingual chatbot interface in both Roman Urdu and English.

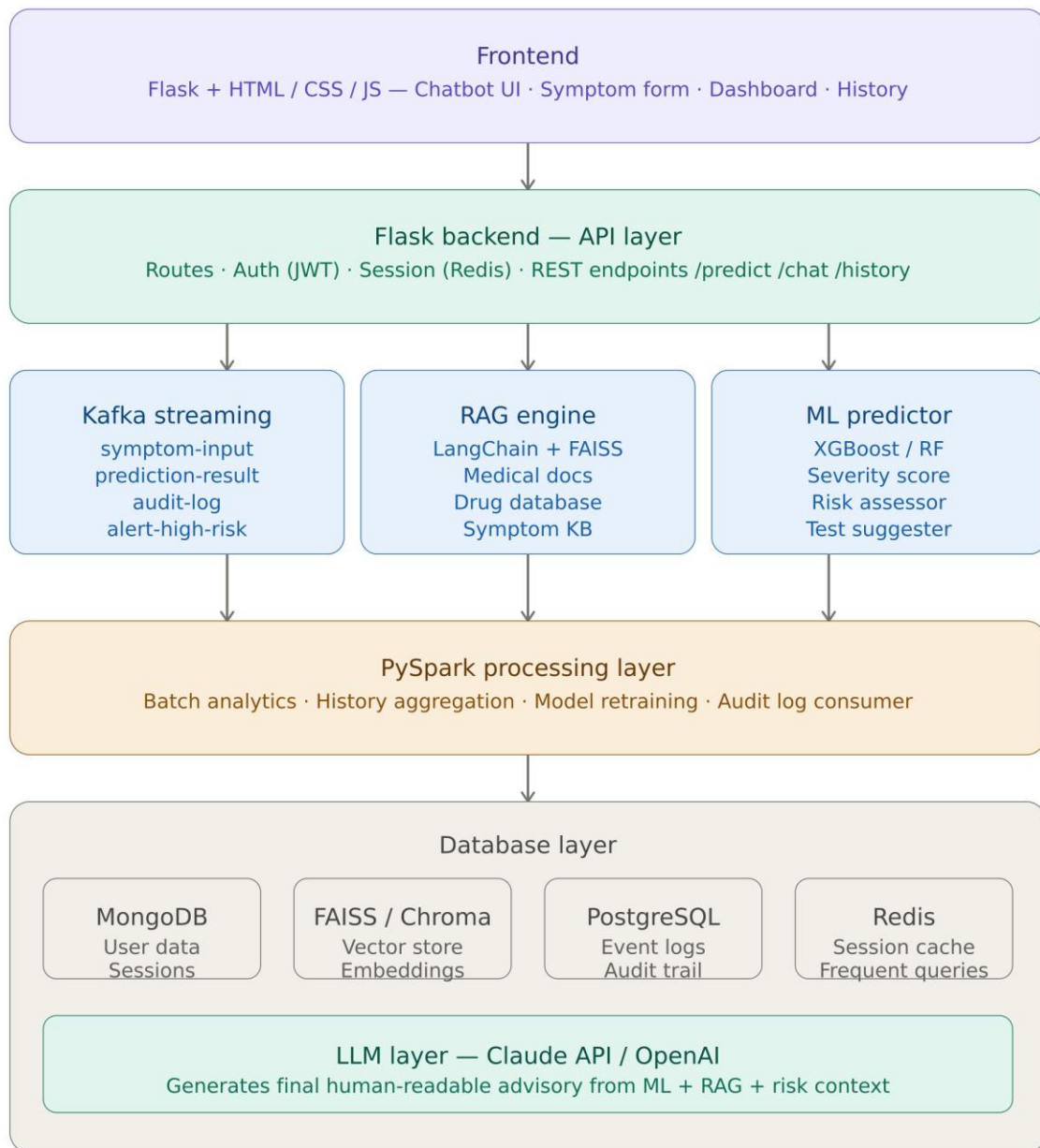
The system leverages Apache Kafka for real-time data streaming, PySpark for big data analytics, LangChain with a vector database for intelligent medical knowledge retrieval, and a Claude/OpenAI LLM for generating human-readable medical advisories.

## Key Capabilities

- **Disease Prediction** — Predicts top 3 diseases with confidence scores using ML (XGBoost/Random Forest)
- **Severity Scoring** — Calculates a 0–100 risk score based on symptoms, duration, and patient age
- **Risk Assessment** — Classifies cases as HIGH / MEDIUM / LOW risk with tailored recommendations
- **RAG-Powered Advice** — Retrieves evidence-based medical knowledge from curated document stores
- **Multilingual Chatbot** — Supports Roman Urdu and English in a conversational interface
- **Big Data Analytics** — PySpark batch jobs aggregate user history and retrain ML models

## 2. System Architecture

The platform follows a layered microservices architecture, with each component communicating via Kafka message topics. Below is the high-level flow:

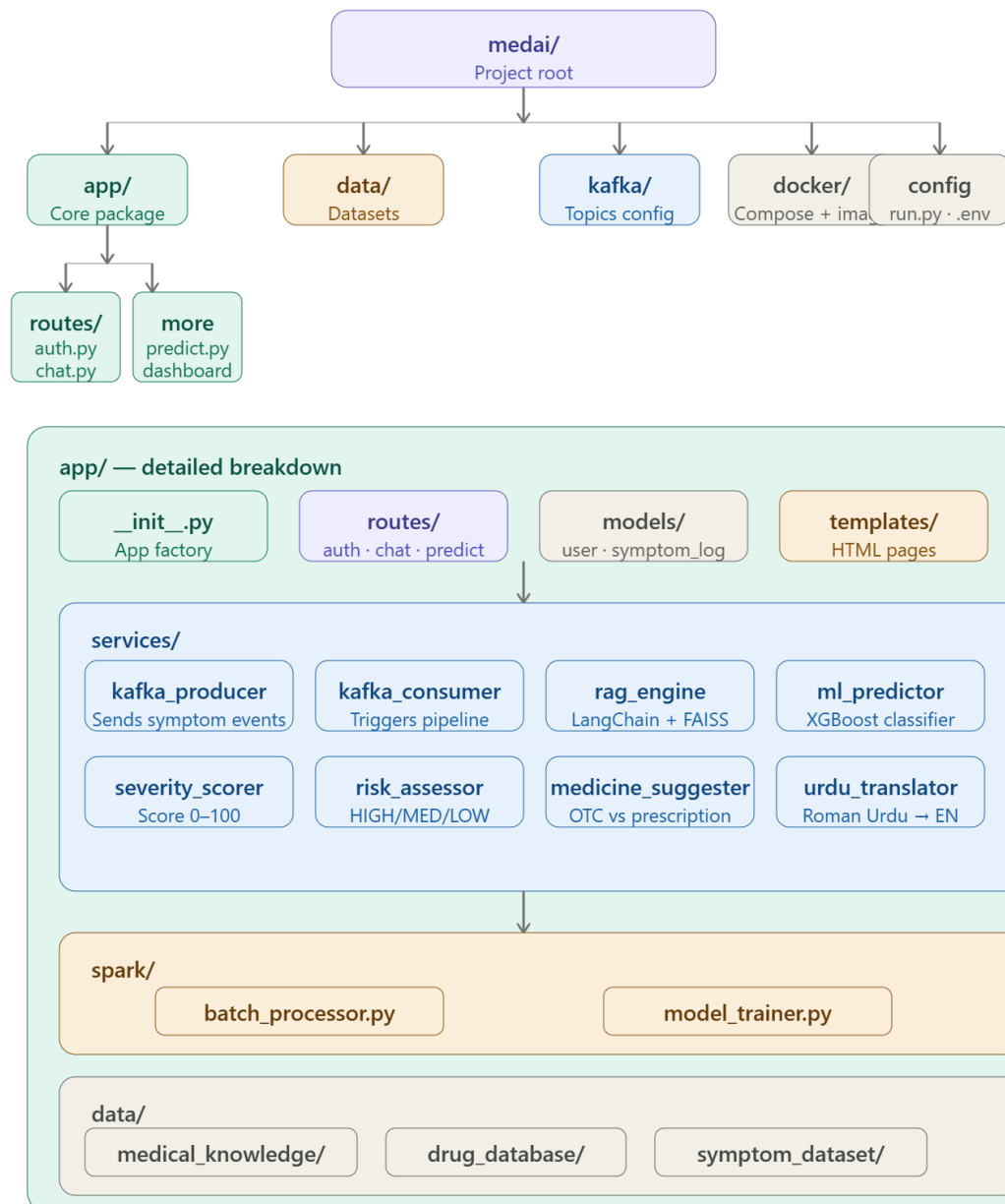


### 3. Technology Stack

| Layer      | Technology                                  |
|------------|---------------------------------------------|
| Frontend   | HTML5, CSS3, JavaScript, Bootstrap 5        |
| Backend    | Python Flask, REST API                      |
| Streaming  | Apache Kafka                                |
| Big Data   | PySpark                                     |
| AI / RAG   | LangChain, FAISS / ChromaDB                 |
| LLM        | Claude API (claude-sonnet-4) or OpenAI      |
| ML Model   | Scikit-learn / XGBoost (symptom classifier) |
| Database   | MongoDB + PostgreSQL + Redis                |
| NLP (Urdu) | Deep Translator / Custom Tokenizer          |
| Deployment | Docker, Docker Compose                      |
| Auth       | Flask-Login + JWT                           |

## 4. Project Structure

The project follows a modular structure separating concerns across routes, services, ML models, and data layers:



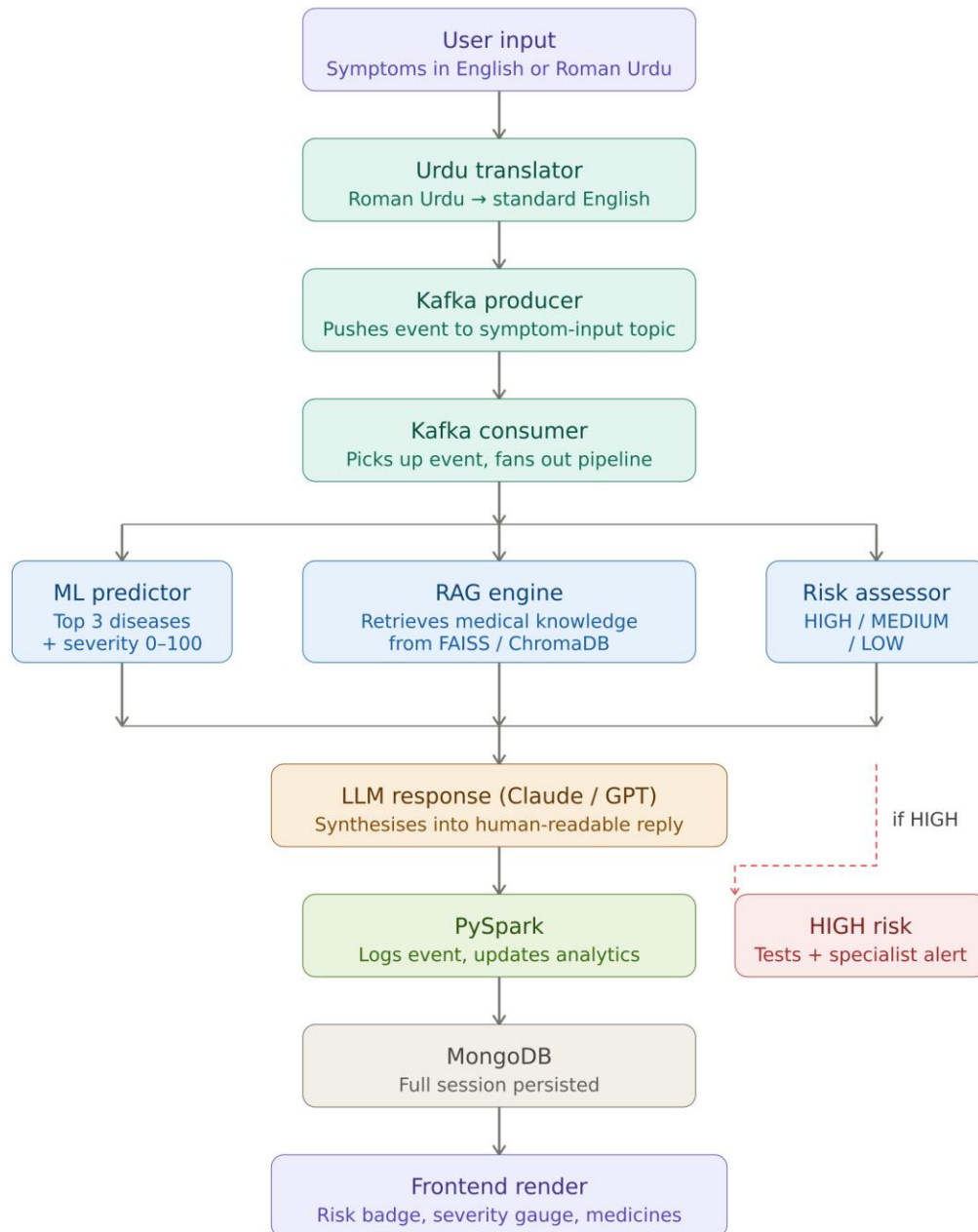
## 5. Data Flow

---

Every user query passes through a multi-stage pipeline before a response is returned:

1. **Step 1 — User Input:** Patient enters symptoms in English or Roman Urdu via the chatbot or form.
2. **Step 2 — Language Detection & Translation:** The Urdu Translator detects the input language and converts Roman Urdu to English using deep-translator and a custom medical dictionary.
3. **Step 3 — Kafka Producer:** The Flask backend pushes the symptom event to the Kafka topic symptom-input for asynchronous processing.
4. **Step 4 — Kafka Consumer Triggers Pipeline:** The consumer picks up the event and fans out to three parallel pipelines: ML Model, RAG Engine, and Risk Assessor.
5. **Step 5a — ML Prediction:** The trained XGBoost/Random Forest model predicts the top 3 diseases with confidence scores and a severity score (0–100).
6. **Step 5b — RAG Retrieval:** LangChain queries the FAISS/ChromaDB vector store to retrieve relevant medical knowledge for the predicted conditions.
7. **Step 5c — Risk Assessment:** Severity score determines risk level (HIGH  $\geq 70$  / MEDIUM 40–69 / LOW  $< 40$ ) with corresponding test and medicine recommendations.
8. **Step 6 — LLM Response Generation:** Claude or GPT synthesises the ML output, RAG context, and risk data into a human-readable, empathetic response.
9. **Step 7 — PySpark Logging:** The complete event is sent to the audit-log Kafka topic for PySpark batch processing and analytics.

10. **Step 8 — Persistence:** MongoDB stores the full session. Redis caches frequent responses for performance.
11. **Step 9 — Frontend Render:** Flask returns a JSON response; the frontend renders the result with risk badge, severity gauge, and recommendations.





## 6. Smart Features Implementation

### 6.1 Severity Score (0–100)

Each symptom carries a base weight. Duration and patient age apply multipliers:

```
SYMPTOM_WEIGHTS = {
    "chest_pain": 30,          "difficulty_breathing": 28,
    "vomiting_blood": 35,      "high_fever": 20,
    "severe_headache": 15,     "fatigue": 8,    "cough": 6
}

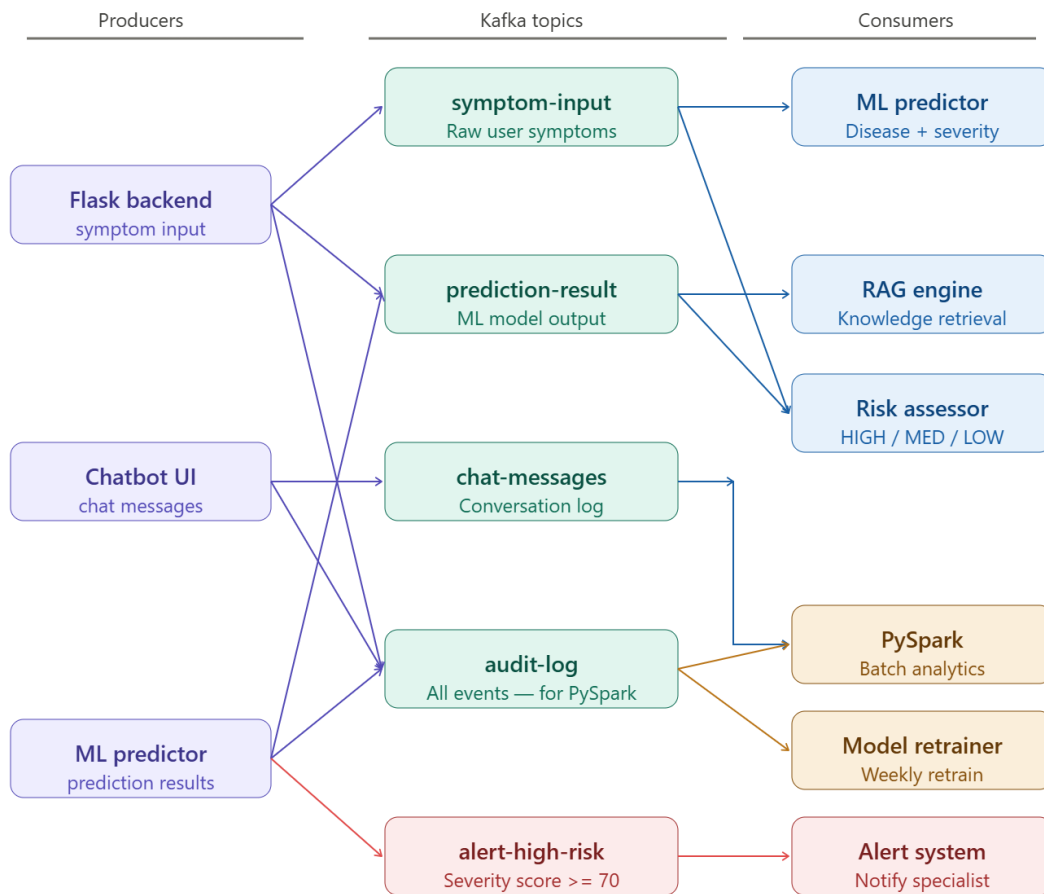
def calculate_severity(symptoms, duration_days, age):
    base = sum(SYMPTOM_WEIGHTS.get(s, 5) for s in symptoms)
    duration_factor = min(1 + (duration_days * 0.1), 2.0)
    age_factor = 1.3 if age > 60 or age < 5 else 1.0
    return min(int(base * duration_factor * age_factor), 100)
```

### 6.2 Risk Assessment Logic

| Severity Score | Risk Level                                                                                 | Recommendations                                                         |
|----------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| ≥ 70           |  HIGH   | CBC, LFT, chest X-ray; consult specialist; prescription-level medicines |
| 40 – 69        |  MEDIUM | Basic blood test; OTC medicines with monitoring                         |
| < 40           |  LOW    | No tests needed; simple OTC suggestions                                 |

### 6.3 Kafka Topics Architecture

```
symptom-input      → raw symptoms from user
prediction-result   → ML model output
chat-messages      → chatbot conversation log
audit-log          → all events (consumed by PySpark)
alert-high-risk    → HIGH severity cases (future: alert doctor)
```



## 6.4 Roman Urdu / English NLP

- Detect input language (English vs Roman Urdu)
- Use deep-translator to convert to English
- Apply custom medical dictionary (e.g., "bukhaar" → "fever", "sar dard" → "headache")
- Process all AI logic in English
- Respond in the same language as the user's input

## 7. 4-Week Development Plan

---

### Week 1 — Foundation & Setup (Days 1–7)

**Goal:** Environment ready, basic Flask app running, Kafka integrated

- **Days 1–2:** Install and configure Python 3.10+, Apache Kafka (Docker), PySpark, MongoDB, Redis. Create virtual environment and install all requirements.
- **Days 3–4:** Build Flask base app: user registration/login (Flask-Login), symptom input form UI, REST API skeleton (/predict, /chat, /history), Redis session management.
- **Days 5–6:** Kafka integration: create topics (symptom-input, prediction-result, audit-log), implement kafka\_producer.py and kafka\_consumer.py, test end-to-end message flow.
- **Day 7:** ML dataset preparation: download symptom-disease dataset from Kaggle, clean with Pandas, train Random Forest / XGBoost classifier, save with joblib.

### Week 2 — Core AI Engine (Days 8–14)

**Goal:** Full prediction pipeline with RAG + risk assessment

- **Days 8–9:** RAG engine setup: collect medical PDFs, chunk with LangChain TextSplitter, embed with sentence-transformers, store in FAISS / ChromaDB, build retrieval chain.
- **Days 10–11:** Disease prediction: build symptom vector pipeline, output top-3 diseases with confidence scores and severity score (0–100).
- **Days 12–13:** Risk assessor and diagnostic test suggester based on severity thresholds.
- **Day 14:** Medicine suggestion: query vector DB for disease, generate dosage/warning info via LLM, distinguish OTC vs prescription based on risk level.

## Week 3 —Smart Features + PySpark (Days 15–21)

**Goal:** Chatbot live, Roman Urdu working, PySpark analytics done

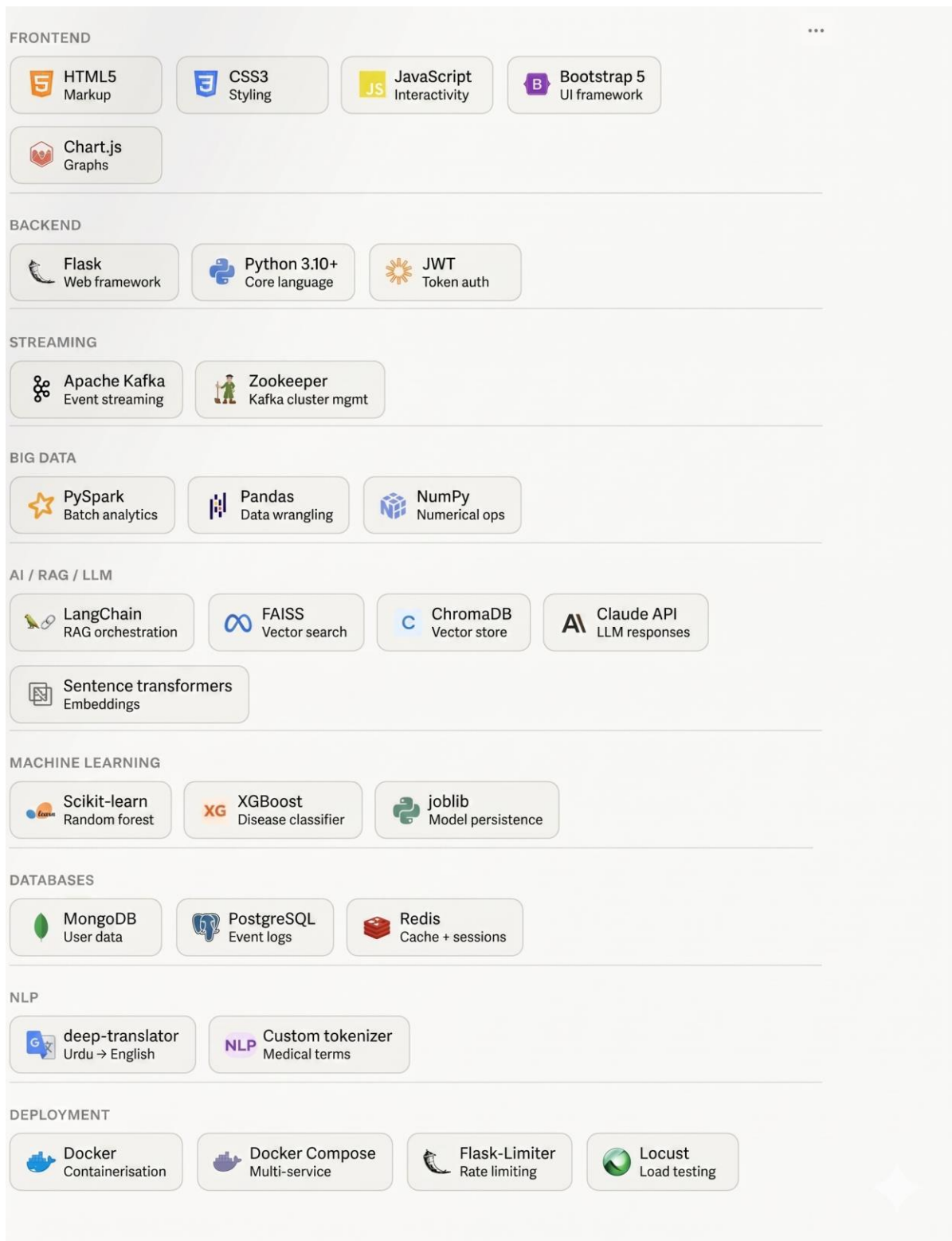
- **Days 15–16:** Multi-turn chatbot with LangChain ConversationBufferMemory; Kafka streams each message for logging.
- **Days 17–18:** Roman Urdu / English NLP pipeline with custom medical dictionary and deep-translator.
- **Day 19:** PySpark batch job (every 6 hours): aggregate symptom logs, generate personal health trend reports, store back to MongoDB.
- **Days 20–21:** Continuous learning pipeline: collect new data, batch retrain with PySpark, A/B test model accuracy, auto-deploy better model.

## Week 4 — Polish, Integration & Deployment (Days 22–28)

**Goal:** Production-ready application deployed

- **Days 22–23:** Full UI/UX polish: Chart.js dashboards, animated severity gauge, risk badge, loading animations, print/download report feature.
- **Days 24–25:** Docker Compose setup with services: flask-app, kafka, zookeeper, pyspark, mongodb, redis, chromadb.
- **Day 26:** Testing: unit tests, integration tests, load test (100 concurrent users via Locust), edge case handling.
- **Day 27:** Security: input sanitization, rate limiting (Flask-Limiter), JWT refresh logic, privacy compliance, medical disclaimers.
- **Day 28:** Final demo prep: record demo video, write README.md, prepare presentation slides, deploy to cloud (Railway / Render / AWS EC2).

# Tools and Technologies Used



## Challenges & Mitigation Strategies

Implementing MediSpark, a real-time healthcare platform, involves several technical and operational hurdles. Below are the key challenges and our strategies to address them:

### Real-Time Data Consistency & Latency

- **Challenge:** Managing asynchronous communication between Flask, Kafka, and PySpark can cause data lag.
- **Mitigation:** We will use Redis for session caching and frequent query storage to ensure instant responses.
- **Mitigation:** Locust load testing in Week 4 will optimize Kafka performance for high traffic.

### NLP Accuracy in Roman Urdu

- **Challenge:** The lack of standardized spelling in Roman Urdu (e.g., "bukhaar" vs "bukhar") can lead to misinterpretation.
- **Mitigation:** We are building a custom medical dictionary and tokenizer to map phonetic Urdu to standard English medical terms.

### Clinical Reliability of RAG & LLM Outputs

- **Challenge:** Large Language Models (LLMs) can "hallucinate" or provide incorrect medical dosages.
- **Mitigation:** We will use a Retrieval-Augmented Generation (RAG) architecture with LangChain and FAISS.
- **Mitigation:** Responses will be strictly grounded in verified data from WHO and DrugBank.

### Big Data Integration & Model Retraining

- **Challenge:** Ensuring that continuous retraining via PySpark actually improves model accuracy.

## 8. Datasets

| Dataset                    | Source       | Use Case               |
|----------------------------|--------------|------------------------|
| Disease Symptom Prediction | Kaggle       | ML model training      |
| DrugBank Open Data         | drugbank.ca  | Medicine suggestions   |
| WHO Disease Profiles       | who.int      | RAG document store     |
| MedQuAD                    | GitHub (NIH) | Q&A RAG pairs          |
| Columbia Medical Corpus    | HuggingFace  | Embeddings fine-tuning |

## 9. Key Dependencies (requirements.txt)

```
flask==3.0.0 flask-login==0.6.3
kafka-python==2.0.2 pyspark==3.5.0
langchain==0.1.0 langchain-community==0.0.20
chromadb==0.4.22 sentence-transformers==2.3.1
scikit-learn==1.4.0 xgboost==2.0.3
pymongo==4.6.1 redis==5.0.1
deep-translator==1.11.4 anthropic==0.18.0
pandas==2.1.4 numpy==1.26.3
joblib==1.3.2 chart.js (CDN)
```

## 10. Project Summary

| Week   | Focus                          | Key Deliverable                              |
|--------|--------------------------------|----------------------------------------------|
| Week 1 | Setup + Kafka + Basic ML       | Running Flask app with Kafka pipeline        |
| Week 2 | RAG + Prediction + Risk Engine | Full AI prediction with medicine suggestions |
| Week 3 | Chatbot + Urdu + PySpark       | Smart features + analytics dashboard         |
| Week 4 | Polish + Docker + Deploy       | Production-ready deployed application        |